DSB6000 - Data Science Strategy & Leadership

NAPA Pickleball Case

Dataset Specification and Release Guide

Addendum to the NAPA Olympic Analytics Platform RFP

Updated for the Student-Facing Dataset Export Contract

Document Version 4.0 | Student Dataset Schema Version 1.6

Document Purpose

This document defines the release package model, delivery format, physical source schema, conceptual data model, key relationships, manifest metadata, and baseline data-quality expectations for the NAPA Olympic Analytics Platform assignment. It is a technical addendum to the NAPA RFP and should be read with the Canvas-managed assignment and milestone requirements.

The physical schemas and release conventions in this version are aligned to the current student-facing Parquet export contract. The files delivered in each dataset release and the accompanying manifest remain the operational source of truth.

1. Executive Overview

NAPA provides synthetic pickleball datasets representing a realistic operational analytics environment containing athletes, clubs, regions, doubles teams, memberships, match participation, match outcomes, game-level scoring, ratings, registrations, assessments, and monthly processing metadata. The datasets support governed data engineering, athlete and partnership evaluation, Olympic roster recommendations, development-pipeline analysis, data-quality management, and executive decision support.

The assignment uses three scale levels—approximately 5,000, 50,000, and 250,000 players—with one consistent student-facing schema. Teams are expected to move between scales and releases through configuration changes rather than code rewrites.

2. Relationship to the Assignment Package

Document	Role
Assignment Overview	Provides the business scenario, consulting competition framing, and high-level delivery sequence.
NAPA RFP	Defines the client-facing problem, scope, outcomes, and proposal expectations.
This Dataset Specification	Defines the provided source data, complete-release package, schema contract, relationships, and baseline quality expectations.
Phase 0 and Milestone Requirements	Define required evidence, Canvas submissions, dates, and grading criteria for each stage.

3. Data Release Strategy

Release	Primary use	Approximate scale	Purpose
5K Development	Milestone 1	5,000 players	Rapid onboarding, Raw landing, Bronze creation, schema capture, configuration, and load validation.
50K Validation	Milestone 2	50,000 players	Silver engineering, data-quality implementation, referential checks, scaling observations, and early analytics.
250K Production	Milestone 3	250,000 players	Production-scale execution, Gold products, final recommendations, tournament candidates, and executive evidence.

- All code must run across the 5K, 50K, and 250K datasets using configuration changes only.
- Final analytical conclusions and roster recommendations must be based on the 250K production dataset.
- The same table names and published column order apply across scale levels.
- A release package may contain a clean variant and, when enabled, a tainted variant with intentionally injected data-quality issues.
- Instructor-only injection manifests and hidden simulation fields are not included in student-facing packages.

4. Release Package Model

A release package is the unit distributed to students. Each package contains one complete historical dataset release for the applicable scale. When data-quality injection is enabled, the package may include both a clean variant and a tainted variant with the same table names, column order, and historical coverage.

<release_name>

Each release is delivered as a complete, independently loadable historical dataset.

4.1 Release Contents

Required file	Purpose
clubs.parquet	Student-facing Parquet table
club_memberships.parquet	Student-facing Parquet table
match_games.parquet	Student-facing Parquet table
match_team_players.parquet	Student-facing Parquet table
match_teams.parquet	Student-facing Parquet table
matches.parquet	Student-facing Parquet table
monthly_batches.parquet	Student-facing Parquet table
player_assessment_history.parquet	Student-facing Parquet table
player_master.parquet	Student-facing Parquet table
player_registrations.parquet	Student-facing Parquet table
regions.parquet	Student-facing Parquet table
team_memberships.parquet	Student-facing Parquet table
teams.parquet	Student-facing Parquet table
manifest.json	Release metadata, validation, schema, and checksum contract

4.2 Historical Coverage and As-of Semantics

Each dataset release contains the complete configured historical period for that release scale. Fact tables contain all rows in the release history, while snapshot-style dimensions represent the latest eligible state as of the release snapshot month.

The monthly_batches table remains part of the dataset because it provides historical reporting periods and supports longitudinal analysis within the single complete release.

5. Manifest Contract

Every dataset release contains manifest.json. It is part of the student-facing package and supports reproducibility, reconciliation, and schema validation.

Manifest field	Description
release_name	Dataset release name.
included_batch_months	ISO month values included in the complete historical release.
included_batch_sequences	Batch sequence numbers included in the release.
snapshot_month	As-of month used for snapshot-style dimensions.
snapshot_end_exclusive	First day after the snapshot month; temporal filters use values earlier than this boundary.
parquet_compression	Compression codec used for the Parquet files.
output_files	Per-file metadata including row count, published columns, schema hash, and checksum.
row_counts	Per-table row counts.
ordered_columns	Published column order by table.
schema_hashes	Schema hash by table.
file_checksums	File checksum by table.
build_timestamp	UTC timestamp when the manifest was written.
student_dataset_schema_version	Published student-facing schema contract version.
validation_status	Release validation status.
validation_summary	Summary of automated release validation results.

6. General Data Conventions

- File format is Parquet; column names use lower snake case.
- Date and datetime values are normalized to ISO-8601 strings before Parquet writing.
- Integer-valued columns are Parquet INT64; UUID values are plain strings; decimal-valued columns are Parquet DECIMAL.
- Primary keys are stable exported identifiers within a generation run.
- Foreign keys point to student-facing tables visible in the same release state.
- Nullable means the exported column may contain null values.
- Hidden simulation values, operational timestamps, job metadata, raw seeds, tournament data, and instructor-only artifacts are excluded.

7. Data Domain and Table Inventory

Domain	Table	Grain
Reference and Geography	regions	One row per referenced geographic region.
Club and Organization	clubs	One row per exported club or facility.
Club and Organization	club_memberships	One row per player-to-club membership interval.

Domain	Table	Grain
Athlete	player_master	One row per player as of the release snapshot.
Athlete	player_registrations	One row per player registration event.
Athlete	player_assessment_history	One row per assessment observation.
Team and Partnership	teams	One row per doubles team.
Team and Partnership	team_memberships	One row per player-to-team membership interval.
Competition and Results	matches	One row per match.
Competition and Results	match_teams	One row per side in a match.
Competition and Results	match_team_players	One row per player on a match side.
Competition and Results	match_games	One row per game within a match.
Temporal Processing	monthly_batches	One row per monthly fact batch.

8. Conceptual Data Model and Relationships

Players are associated with regions, clubs, registrations, assessments, teams, and match participation. Persistent doubles teams are modeled separately from individual players. Match detail is represented through matches, match sides, player participation, and game scores. Monthly batches provide the temporal frame for facts and snapshot evaluation.

Child reference	Parent key	Nullable
clubs.region_id	regions.id	No
club_memberships.player_id	player_master.player_id	No
club_memberships.club_id	clubs.id	No
match_games.match_id	matches.id	No
match_team_players.match_team_id	match_teams.id	No
match_team_players.player_id	player_master.player_id	No
match_teams.match_id	matches.id	No
match_teams.team_id	teams.id	No
matches.region_id	regions.id	Yes
matches.batch_id	monthly_batches.id	No
matches.winning_team_id	teams.id	Yes

Child reference	Parent key	Nullable
player_assessment_history.player_id	player_master.player_id	No
player_assessment_history.batch_id	monthly_batches.id	No
player_master.home_region_id	regions.id	Yes
player_registrations.player_id	player_master.player_id	No
player_registrations.batch_id	monthly_batches.id	No
player_registrations.assigned_region_id	regions.id	Yes
team_memberships.team_id	teams.id	No
team_memberships.player_id	player_master.player_id	No

When matches.winning_team_id is populated, it must match one of the two match_teams.team_id values associated with that match.

9. Source Filtering and Snapshot Projection

Output table	Export filter
clubs	Clubs founded before the release snapshot boundary, including all clubs referenced by included memberships.
club_memberships	Memberships for included players and clubs with joined_date before the release snapshot boundary; left dates after the boundary are projected to null.
match_games	Games whose match_id belongs to an included match.
match_team_players	Player rows whose match_team_id belongs to an included match side.
match_teams	Match sides whose match_id belongs to an included match.
matches	All matches whose batch_id belongs to the historical batch window included in the release.
monthly_batches	All completed monthly batches for the selected generation run and configured historical period.
player_assessment_history	Assessment rows tied to included historical batches and included players.
player_master	Players registered before the release snapshot boundary, with the latest eligible public rating state before that boundary.
player_registrations	Registration rows tied to included historical batches and included players.
regions	All regions referenced by included players, clubs, registrations, or matches.

Output table	Export filter
team_memberships	Memberships for included teams and players with joined_date before the snapshot boundary; left dates after the boundary are projected to null.
teams	Teams formed before the snapshot boundary; dissolution dates after the boundary are projected to null.

10. Excluded Source Data

- players as a standalone output; current player state is published as player_master.
- player_rating_history as a standalone output; the latest eligible public rating is folded into player_master.
- Generation parameters, job status, runtime metrics, and export bookkeeping.
- Raw seed files and normalized seed tables.
- Hidden expected-performance, prediction, noise, chemistry, persistence, regional-strength, or related simulation fields.
- Tournament simulation tables.
- Instructor-only data-quality injection manifests.

11. Physical Table Dictionary

11.1 clubs

Business description: Club, facility, and organization dimension.

Grain: One row per exported club.

Source: clubs

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Club identifier.
club_name	STRING	No	None	Club or facility name.
region_id	INT64	No	regions.id	Region where the club is located.
club_type	STRING	Yes	None	Club classification.
competitiveness_level	STRING	Yes	None	Competitive segment assigned to the club.
member_capacity	INT64	Yes	None	Approximate member capacity.
founding_date	STRING (ISO date)	Yes	None	Club founding date.
indoor_court_count	INT64	Yes	None	Number of indoor courts.
outdoor_court_count	INT64	Yes	None	Number of outdoor courts.

Excluded source columns: generation_run_id, created_at, updated_at.

11.2 club_memberships

Business description: Player-to-club membership intervals.

Grain: One row per exported club membership interval.

Source: club_memberships

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Club membership identifier.
player_id	INT64	No	player_master.player_id	Member player.
club_id	INT64	No	clubs.id	Club joined by the player.
membership_type	STRING	Yes	None	Membership category.
joined_date	STRING (ISO date)	No	None	Membership start date.
left_date	STRING (ISO date)	Yes	None	Membership end date; future dates project to null.
is_primary	BOOL	Yes	None	Whether this is the player's primary club membership.

Excluded source columns: generation_run_id, created_at, updated_at.

11.3 match_games

Business description: Game-level scoring rows inside matches.

Grain: One row per exported game.

Source: match_games

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Match game identifier.
match_id	INT64	No	matches.id	Match containing the game.
game_number	INT64	No	None	Game sequence number.
team_one_score	INT64	No	None	Score for side 1.
team_two_score	INT64	No	None	Score for side 2.
winning_team_number	INT64	No	None	Winning side number: 1 or 2.
target_score	INT64	Yes	None	Target score.
win_by	INT64	Yes	None	Required winning margin.
actual_team_one_score_share	DECIMAL (scale 4)	Yes	None	Team one share of total points.

Excluded source columns: expected_team_one_score_share, expected_team_one_score, expected_team_two_score, score_noise_factor, created_at, updated_at.

11.4 match_team_players

Business description: Player participation within match sides.

Grain: One row per exported player on a match side.

Source: match_team_players

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Row identifier.
match_team_id	INT64	No	match_teams.id	Match side containing the player.
player_id	INT64	No	player_master.player_id	Participating player.
player_position	INT64	Yes	None	Player order on the side.
player_rating_at_match	DECIMAL (scale 3)	Yes	None	Public player rating at match time.

Excluded source columns: created_at, updated_at.

11.5 match_teams

Business description: Side-level facts within matches.

Grain: One row per side in an exported match.

Source: match_teams

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Match side identifier.
match_id	INT64	No	matches.id	Match containing this side.
team_number	INT64	No	None	Side number within the match.
team_id	INT64	No	teams.id	Persistent team identity, exported from source_team_id.
team_score	INT64	No	None	Match-level score for the side.
average_team_rating	DECIMAL (scale 3)	Yes	None	Average public rating for players on the side.

Excluded source columns: expected_win_probability, pairing_source, source_team_id, created_at, updated_at.

11.6 matches

Business description: Match-level facts.

Grain: One row per exported match.

Source: matches

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Match identifier.

Column	Type	Nullable	Foreign key	Description
match_date	STRING (ISO date)	No	None	Match date.
region_id	INT64	Yes	regions.id	Region where the match occurred.
match_type	STRING	No	None	Match type.
court_type	STRING	Yes	None	Court context.
match_format	STRING	Yes	None	Match format descriptor.
winning_team_id	INT64	Yes	teams.id	Persistent team id for the winning side.
total_points_played	INT64	Yes	None	Total points across all games.
batch_id	INT64	No	monthly_batches.id	Monthly batch for the match.

Excluded source columns: tournament_id, predicted_winning_team_number, predicted_win_probability, expected_competitiveness, simulation_noise_factor, created_at, updated_at.

11.7 monthly_batches

Business description: Monthly reporting periods in the release fact window.

Grain: One row per exported monthly batch.

Source: monthly_batches

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Monthly batch identifier.
batch_month	STRING (ISO date)	No	None	First day of the reporting month.
batch_sequence	INT64	No	None	Sequence within the generation run.
batch_type	STRING	No	None	Timeline classification.
active_player_count_start	INT64	Yes	None	Active player count at month start.
new_player_count	INT64	Yes	None	Players added during the month.
active_player_count_end	INT64	Yes	None	Active player count at month end.
match_count_generated	INT64	Yes	None	Matches generated for the month.
rating_update_count	INT64	Yes	None	Rating rows generated.
assessment_update_count	INT64	Yes	None	Assessment rows generated.

Excluded source columns: generation_run_id, processing_status, started_at, completed_at, error_message, created_at, updated_at.

11.8 player_assessment_history

Business description: Player assessment observations.

Grain: One row per exported assessment observation.

Source: player_assessment_history

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Assessment identifier.
player_id	INT64	No	player_master.player_id	Assessed player.
assessment_date	STRING (ISO date)	No	None	Assessment date.
assessment_type	STRING	No	None	Assessment category.
assessment_value	DECIMAL (scale 3)	Yes	None	Assessment value.
confidence_score	DECIMAL (scale 3)	Yes	None	Assessment confidence.
derived_from_matches	INT64	Yes	None	Matches used in the assessment.
batch_id	INT64	No	monthly_batches.id	Monthly batch for the row.

Excluded source columns: created_at.

11.9 player_master

Business description: Player dimension with current public rating state as of the release snapshot.

Grain: One row per exported player.

Source: players plus latest eligible player_rating_history row

Column	Type	Nullable	Foreign key	Description
player_id	INT64	No	None	Player identifier from players.id.
external_player_key	STRING (UUID text)	No	None	Stable external player identifier.
first_name	STRING	No	None	Player first name.
last_name	STRING	No	None	Player last name.
gender	STRING	Yes	None	Player gender.
birth_date	STRING (ISO date)	No	None	Player birth date.
dominant_hand	STRING	Yes	None	Dominant hand.
home_region_id	INT64	Yes	regions.id	Player home region.

Column	Type	Nullable	Foreign key	Description
country_code	STRING	Yes	None	Home-region country code resolved from regions.country_code.
registration_date	STRING (ISO date)	No	None	Date the player entered the population.
player_status	STRING	No	None	Player lifecycle status.
rating	DECIMAL (scale 3)	Yes	None	Compatibility alias of rating_value for ranking-oriented consumers.
rating_value	DECIMAL (scale 3)	Yes	None	Latest public rating before the snapshot boundary.
confidence_score	DECIMAL (scale 3)	Yes	None	Latest rating confidence.
volatility_score	DECIMAL (scale 3)	Yes	None	Latest rating volatility.
global_percentile	DECIMAL (scale 2)	Yes	None	Latest global percentile.
match_count_used	INT64	Yes	None	Match count reflected in the latest rating.
rating_date	STRING (ISO date)	Yes	None	Effective date of the latest rating.
rating_batch_id	INT64	Yes	None	Source batch for the latest rating.
snapshot_month	STRING (ISO date)	No	None	Release snapshot month on every row.

Excluded source columns: players.id is renamed player_id; initial_skill_seed, generation_run_id, created_at, updated_at. player_rating_history is not separately published.

11.10 player_registrations

Business description: Player onboarding events.

Grain: One row per exported registration event.

Source: player_registrations

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Registration identifier.
player_id	INT64	No	player_master.player_id	Registered player.
batch_id	INT64	No	monthly_batches.id	Monthly batch for the event.
registration_month	STRING (ISO date)	No	None	Registration reporting month.

Column	Type	Nullable	Foreign key	Description
registration_source	STRING	Yes	None	Acquisition or registration source.
assigned_region_id	INT64	Yes	regions.id	Region assigned at registration.
initial_rating_value	DECIMAL (scale 3)	Yes	None	Initial public rating.
initial_confidence_score	DECIMAL (scale 3)	Yes	None	Initial confidence score.

Excluded source columns: created_at.

11.11 regions

Business description: Geographic markets used by players, clubs, registrations, and matches.

Grain: One row per referenced region.

Source: regions

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Region identifier.
country_code	STRING	No	None	Country code, such as US or CA.
region_type	STRING	Yes	None	Region classification.
region_name	STRING	No	None	Human-readable region name.
state_province_code	STRING	Yes	None	State or province code.
population	INT64	Yes	None	Approximate market population.
latitude	DECIMAL (scale 6)	Yes	None	Latitude.
longitude	DECIMAL (scale 6)	Yes	None	Longitude.

Excluded source columns: selection_probability, competitiveness_multiplier, created_at, updated_at.

11.12 team_memberships

Business description: Player-to-team membership intervals.

Grain: One row per exported team membership interval.

Source: team_memberships

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Team membership identifier.
team_id	INT64	No	teams.id	Team joined by the player.

Column	Type	Nullable	Foreign key	Description
player_id	INT64	No	player_master.player_id	Player on the team.
player_position	INT64	No	None	Player order on the team.
joined_date	STRING (ISO date)	No	None	Membership start date.
left_date	STRING (ISO date)	Yes	None	Membership end date; future dates project to null.

Excluded source columns: created_at, updated_at.

11.13 teams

Business description: Doubles team dimension.

Grain: One row per exported team.

Source: teams

Column	Type	Nullable	Foreign key	Description
id	INT64	No	None	Team identifier.
team_type	STRING	No	None	Team identity type: competitive or ad_hoc.
team_division	STRING	No	None	Competition category: mens_doubles, womens_doubles, mixed_doubles, or open_doubles.
team_status	STRING	No	None	Lifecycle status projected as of the release snapshot.
country_code	STRING	Yes	None	Team country code when player-region country is known.
formation_date	STRING (ISO date)	No	None	Team formation date.
dissolution_date	STRING (ISO date)	Yes	None	Dissolution date; future dates project to null.

Excluded source columns: chemistry_score, persistence_probability, generation_run_id, created_at, updated_at.

12. Medallion Architecture Expectations

Layer	Purpose	Expected treatment
Raw	Preserve the delivered complete source release.	Retain original Parquet files and manifest metadata; document the dataset scale and clean or tainted variant.
Bronze	Managed tables aligned to the physical source contract.	Preserve source columns, add ingestion metadata where useful, and reconcile to manifest row counts/checksums.

Layer	Purpose	Expected treatment
Silver	Reliable business entities.	Apply keys, data types, deduplication, referential validation, temporal logic, and documented business transformations.
Gold	Decision-ready analytical products.	Develop rankings, scorecards, partnership measures, development candidates, quality confidence, and executive outputs.

13. Data Quality and Validation Expectations

The student-facing dataset is designed to resemble a realistic operational environment. A tainted variant may contain intentionally injected defects. Teams should distinguish package integrity failures from quality findings that are valid inputs to their engineering and analytical work.

Rule area	Minimum expectation	Typical severity	Business impact
Release integrity	All required files and manifest are present; schema hashes and checksums can be reconciled.	High	Missing or corrupted artifacts prevent reliable ingestion.
Primary keys	Identifiers expected to be unique and non-null.	High	Duplicates corrupt joins, counts, and rankings.
Referential integrity	Published foreign keys should resolve within the release state.	High	Broken relationships undermine entity and performance analysis.
Match structure	Each match should have two sides; side numbering and winner references should be coherent.	High	Invalid structure affects outcomes and scorecards.
Game integrity	Scores should be non-negative and winner indicators should align with scores.	Medium	Invalid game rows distort consistency and margin measures.
Temporal consistency	Registration, formation, membership, rating, match, and dissolution dates should progress logically.	Medium	Temporal defects distort trends and development analysis.
Historical completeness	All configured historical batches should be represented consistently, and snapshot-style dimensions should reflect the stated as-of month.	High	Incomplete history or incorrect as-of logic can distort trends, ratings, and roster recommendations.
Tournament eligibility	Submitted team IDs must exist and resolve to valid memberships and divisions.	High	Invalid teams cannot enter the instructor tournament.

14. Analytical Requirements Supported by the Dataset

Outcome	Relevant source inputs	Expected use
Olympic roster recommendations	player_master, teams, team_memberships, matches, match_teams, match_team_players, match_games	Defensible men's, women's, and mixed doubles recommendations for Canada and the United States.
National player rankings	player_master, match_team_players, match_teams, matches, match_games	Explainable player-level rankings distinct from team effectiveness.
Development pipeline	player_master, player_assessment_history, player_registrations, match participation	Identify emerging and improving athletes and partnerships.
Partnership analysis	teams, team_memberships, match_teams, match_team_players, matches, match_games	Evaluate continuity, performance, consistency, and risk.
Tournament candidates	teams, team_memberships, player_master, Gold scorecards	Submit valid existing team IDs by country and division; no invented pairings.
Quality confidence	All source files, manifest, and quality outputs	Explain how quality findings affect confidence in recommendations.

15. Student Guardrails and Assumptions

- The data is fictional and synthetic but should be treated as a realistic NAPA operational source.
- The 5K, 50K, and 250K packages are scale levels of the same analytical environment.
- Clean and tainted variants use the same table names, schema contract, and column order.
- Final recommendations must use the 250K production dataset.
- Tournament submissions must use existing teams and valid team IDs; ad-hoc pairings may only be selected when they already exist as team records.
- Ratings alone are insufficient; teams should consider partnership performance, consistency, competition context, confidence, trajectory, and quality limitations.
- Delivered files and manifest metadata are the operational source of truth. Differences from narrative examples should be documented and handled using the delivered contract.

Appendix A. Release Processing Checklist

Step	Complete dataset release
1. Identify release	Read manifest; confirm release name, dataset scale, snapshot month, and schema version.
2. Validate package	Check required files, schema hashes, checksums, and row counts.
3. Load Raw/Bronze	Load all published tables as a complete release.
4. Reconcile	Compare loaded row counts and schemas with manifest metadata.

5. Validate relationships	Run primary-key, foreign-key, match-structure, team-membership, and temporal checks.

Appendix B. Key Changes from the Prior Specification

- Formalized student dataset schema version 1.6.
- Added clean and tainted release variants and instructor-only injection-manifest exclusion.
- Clarified that each student dataset is delivered as one complete, independently loadable historical release.
- Added manifest.json as a required release artifact with row counts, ordered columns, schema hashes, checksums, and validation status.
- Confirmed player_master as the published player table and removed standalone players and player_rating_history outputs.
- Added match_teams.team_id as the persistent team reference.
- Added teams.team_division and clarified team_type as competitive or ad_hoc.
- Updated exact Parquet logical types, nullability, foreign keys, source filtering, and excluded hidden columns.
- Clarified snapshot projection for future membership end dates and team dissolution dates.